Mobile Demo Script Explanation -- Fortnite (Exploitation, Challenge 03)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: Both the code file and its expected-hash file are writable with no signature tying them together -- overwrite both with a matching pair.

Background -- what the terms below mean

What does "Exploitation" mean, as a category? An **Exploitation** challenge gives you a real target Android app with an actual security mistake in it, and asks you to write a second, small app of your own that takes advantage of that mistake to get the flag.

What is a vulnerability? A mistake in how software was built that lets someone do something the developer never intended -- here, get the flag without going through the app the normal, intended way.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**) -- to find the mistake in the first place.

What is a package name? Every Android app has a unique ID string (like org.naija.fdexploit). Android refuses to install two different apps that both use the exact same ID, which occasionally trips up a first submission attempt if the exploit app accidentally reuses the target's own package name.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/03 Fortnite\Original Target APK.apk
```

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 03 Fortnite.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

TOCTOU / no real integrity protection: verifyAndRunCode() hash-checks fortnite.dex against sign.dat, but both files sit on world-writable external storage with no signature tying them together -- we overwrite both with our own matching (malicious dex, its own real SHA-256) pair, no race needed. First payload attempt crashed with ClassNotFoundException because DexClassLoader's parent classloader (getSystemClassLoader()) can't see the target's own classes; fixed by walking android.app.ActivityThread's live activity table via reflection to find the real MainActivity instance.

Files to open

1. Open the target APK in jadx-gui:

C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/03 Fortnite\Original Target APK.apk

2. MainActivity.java:

C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/03 Fortnite\Android Studio Project\app\src

This is our own code -- open directly in Android Studio or any text editor.

What to say -- explanation of the exploit, step by step

1. I decompiled the target and found verifyAndRunCode() hash-checks fortnite.dex against sign.dat, but both files sit on world-writable external storage with no signature tying them together.

2. I overwrote both files with my own malicious dex and its own real SHA-256, since no race condition was even needed.

3. My first payload attempt crashed with ClassNotFoundException, because DexClassLoader's parent classloader can't see the target's own classes; I fixed this by walking android.app.ActivityThread's live activity table via reflection to find the real MainActivity instance.

4. I resubmitted and the platform's analysis log confirmed the payload ran and returned the flag.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log." This is backed by a real screenshot of that exact submission page (see **Real Screenshot: Live Platform Submission Page** in this challenge's Report.pdf), not just a claim -- worth pointing to directly if an emulator isn't running.

Result

The flag this challenge produced:

MOBISEC{players_gonna_play_mobisec_hackers_gonna_mobisec_it_up}

Source: Verbatim from /apk/81743 analysis log

The real, verbatim output this run produced:

```
File downloaded from http://10.0.2.2:31337/fortnite to /storage/emulated/0/fortnite.dex
File downloaded from http://10.0.2.2:31337/sign to /storage/emulated/0/sign.dat
PAYLOAD RUNNING - FLAG: MOBISEC{players_gonna_play_mobisec_hackers_gonna_mobisec_it_up}
```